

EXPERIMENT NO. 10

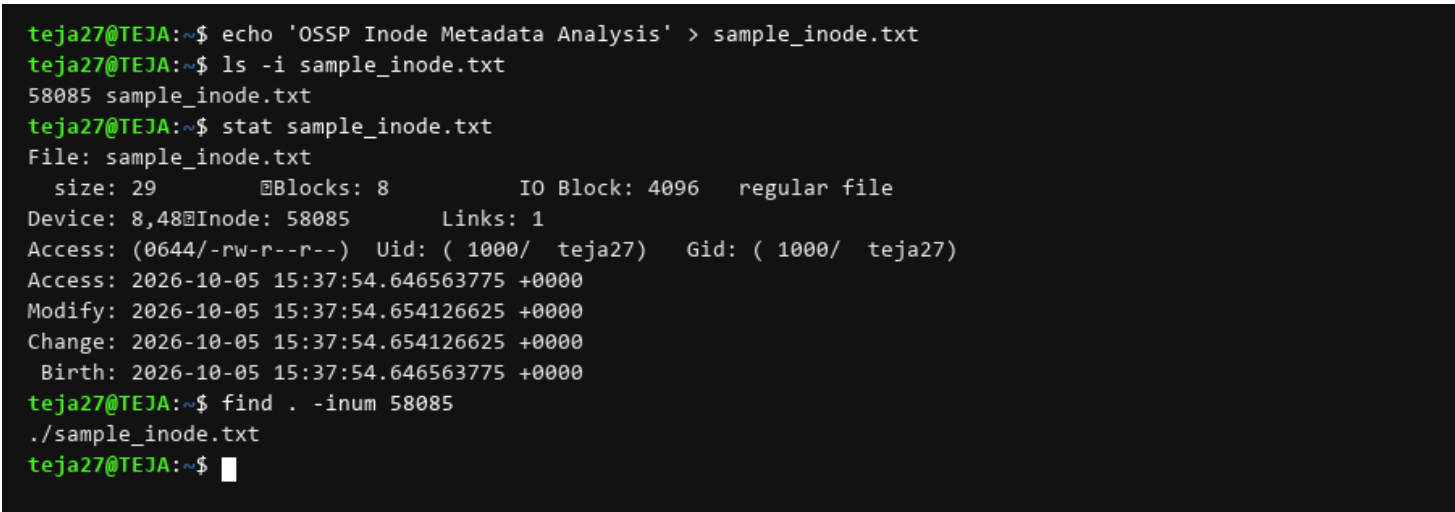
Task 1: Investigate Inode Structures using ls -li, stat, and find

Inspect filesystem inode metadata including inode indices, hard link counters, block allocations, and timestamps.

Commands used:

```
# Inode Structure Investigation Commands
echo "OSSP Inode Metadata Analysis" > sample_inode.txt
ls -li sample_inode.txt
stat sample_inode.txt
INODE_NUM=$(ls -li sample_inode.txt | awk '{print $1}')
find . -inum $INODE_NUM
```

Output screenshot:



```
teja27@TEJA:~$ echo 'OSSP Inode Metadata Analysis' > sample_inode.txt
teja27@TEJA:~$ ls -li sample_inode.txt
58085 sample_inode.txt
teja27@TEJA:~$ stat sample_inode.txt
File: sample_inode.txt
  size: 29      Blocks: 8          IO Block: 4096   regular file
Device: 8,48 Inode: 58085         Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/   teja27)   Gid: ( 1000/   teja27)
Access: 2026-10-05 15:37:54.646563775 +0000
Modify: 2026-10-05 15:37:54.654126625 +0000
Change: 2026-10-05 15:37:54.654126625 +0000
 Birth: 2026-10-05 15:37:54.646563775 +0000
teja27@TEJA:~$ find . -inum 58085
./sample_inode.txt
teja27@TEJA:~$
```

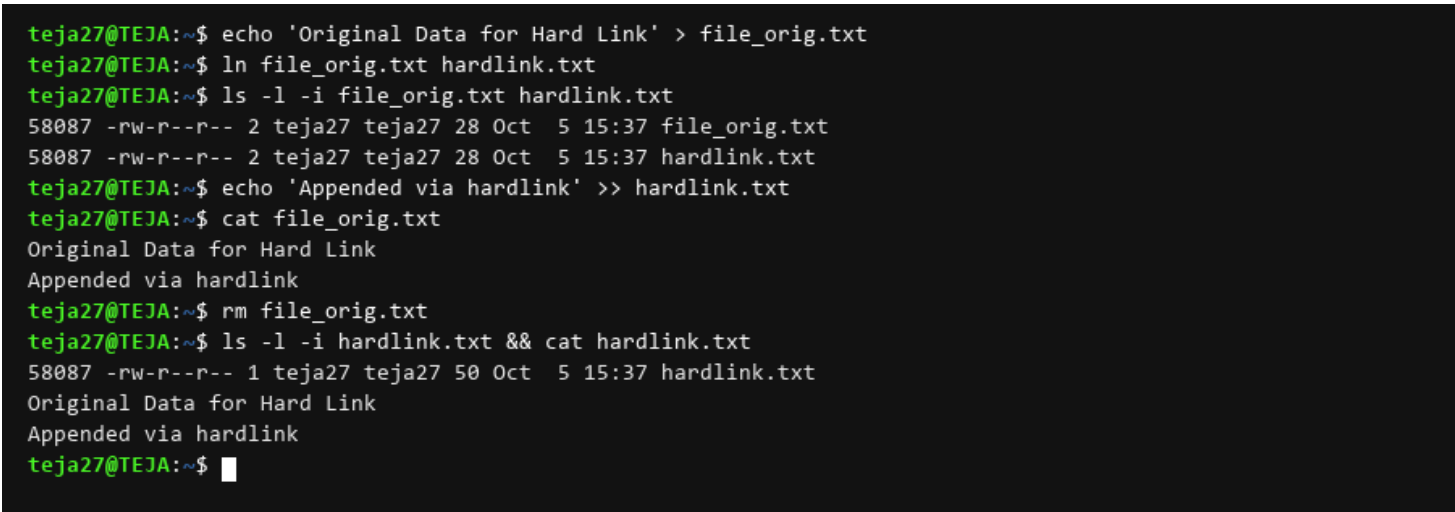
Task 2: Create and Investigate Hard Links

Demonstrate hard link mechanics: shared inode identity, link count increment/decrement, and persistence after primary deletion.

Commands used:

```
# Create and Investigate Hard Links
echo "Original Data for Hard Link" > file_orig.txt
ln file_orig.txt hardlink.txt
ls -l -i file_orig.txt hardlink.txt
echo "Appended via hardlink" >> hardlink.txt
cat file_orig.txt
rm file_orig.txt
ls -l -i hardlink.txt
cat hardlink.txt
```

Output screenshot:



```
teja27@TEJA:~$ echo 'Original Data for Hard Link' > file_orig.txt
teja27@TEJA:~$ ln file_orig.txt hardlink.txt
teja27@TEJA:~$ ls -l -i file_orig.txt hardlink.txt
58087 -rw-r--r-- 2 teja27 teja27 28 Oct  5 15:37 file_orig.txt
58087 -rw-r--r-- 2 teja27 teja27 28 Oct  5 15:37 hardlink.txt
teja27@TEJA:~$ echo 'Appended via hardlink' >> hardlink.txt
teja27@TEJA:~$ cat file_orig.txt
Original Data for Hard Link
Appended via hardlink
teja27@TEJA:~$ rm file_orig.txt
teja27@TEJA:~$ ls -l -i hardlink.txt && cat hardlink.txt
58087 -rw-r--r-- 1 teja27 teja27 50 Oct  5 15:37 hardlink.txt
Original Data for Hard Link
Appended via hardlink
teja27@TEJA:~$
```

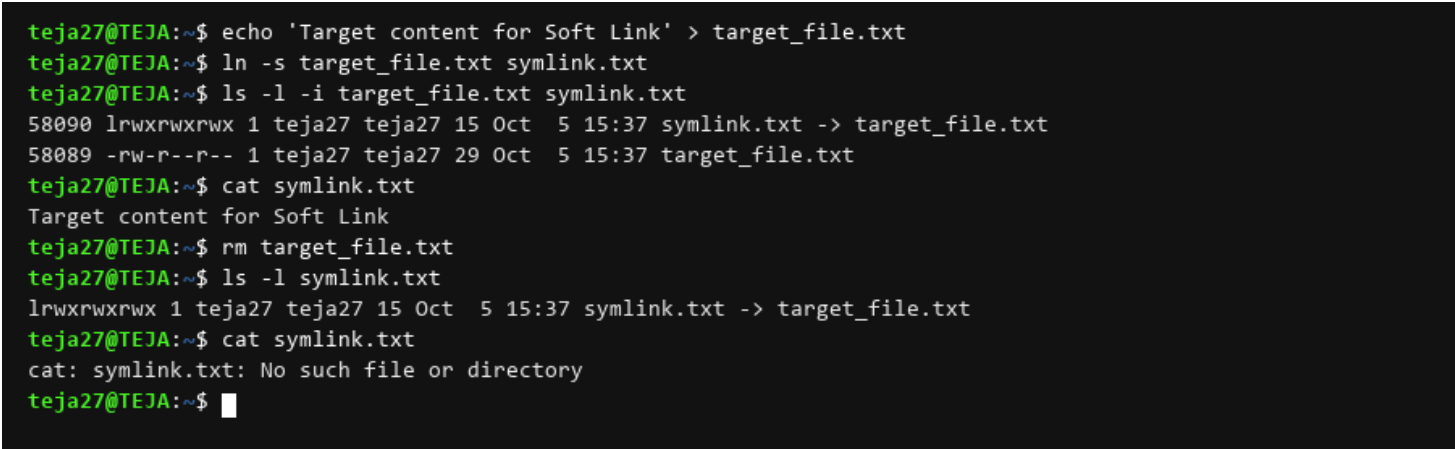
Task 3: Create and Investigate Symbolic / Soft Links

Analyze symbolic links: distinct inode allocation, path redirection, and dangling symlink behavior upon target removal.

Commands used:

```
# Create and Investigate Symbolic (Soft) Links
echo "Target content for Soft Link" > target_file.txt
ln -s target_file.txt symlink.txt
ls -l -i target_file.txt symlink.txt
cat symlink.txt
rm target_file.txt
ls -l symlink.txt
cat symlink.txt
```

Output screenshot:



```
teja27@TEJA:~$ echo 'Target content for Soft Link' > target_file.txt
teja27@TEJA:~$ ln -s target_file.txt symlink.txt
teja27@TEJA:~$ ls -l -i target_file.txt symlink.txt
58090 lrwxrwxrwx 1 teja27 teja27 15 Oct  5 15:37 symlink.txt -> target_file.txt
58089 -rw-r--r-- 1 teja27 teja27 29 Oct  5 15:37 target_file.txt
teja27@TEJA:~$ cat symlink.txt
Target content for Soft Link
teja27@TEJA:~$ rm target_file.txt
teja27@TEJA:~$ ls -l symlink.txt
lrwxrwxrwx 1 teja27 teja27 15 Oct  5 15:37 symlink.txt -> target_file.txt
teja27@TEJA:~$ cat symlink.txt
cat: symlink.txt: No such file or directory
teja27@TEJA:~$
```

Task 4: Implement File Reading using mmap()

Maps file pages directly into user address space via mmap(PROT_READ, MAP_PRIVATE) for zero-copy file inspection.

Complete code:

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>

int main() {
    const char *filepath = "mmap_read_input.txt";

    /* Create sample file */
    FILE *fp = fopen(filepath, "w");
    fprintf(fp, "Memory-Mapped File I/O: High performance zero-copy reading using mmap().\n");
    fclose(fp);

    int fd = open(filepath, O_RDONLY);
    if (fd < 0) { perror("open"); return 1; }

    struct stat sb;
    if (fstat(fd, &sb) < 0) { perror("fstat"); close(fd); return 1; }

    /* Map entire file into virtual memory space (read-only) */
    char *mapped = mmap(NULL, sb.st_size, PROT_READ, MAP_PRIVATE, fd, 0);
    if (mapped == MAP_FAILED) { perror("mmap"); close(fd); return 1; }

    printf("[mmap Read] File size: %ld bytes\n", (long)sb.st_size);
    printf("[mmap Read] Direct virtual memory read contents:\n---\n");
    fwrite(mapped, 1, sb.st_size, stdout);
    printf("---\n");

    /* Cleanup */
    munmap(mapped, sb.st_size);
    close(fd);
    printf("[mmap Read] Unmapped memory and closed file descriptor successfully.\n");
    return 0;
}
```

Output screenshot:

```
teja27@TEJA:~$ nano exp10_task4.c
teja27@TEJA:~$ gcc exp10_task4.c -o exp10_task4
teja27@TEJA:~$ ./exp10_task4
[mmap Read] File size: 73 bytes
[mmap Read] Direct virtual memory read contents:
---
Memory-Mapped File I/O: High performance zero-copy reading using mmap().
---
[mmap Read] Unmapped memory and closed file descriptor successfully.
teja27@TEJA:~$
```

Task 5: Implement File Writing using mmap() and msync()

Modifies persistent file data via memory pointers with MAP_SHARED and flushes pages using msync().

Complete code:

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <string.h>

#define FILESIZE 128

int main() {
    const char *filepath = "mmap_write_output.txt";
    int fd = open(filepath, O_RDWR | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) { perror("open"); return 1; }

    /* Set the file to fixed size */
    if (ftruncate(fd, FILESIZE) < 0) { perror("ftruncate"); close(fd); return 1; }

    /* Map file with write permissions and MAP_SHARED so changes sync to disk */
    char *mapped = mmap(NULL, FILESIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
    if (mapped == MAP_FAILED) { perror("mmap"); close(fd); return 1; }

    printf("[mmap Write] Memory mapped at address %p with size %d\n", (void*)mapped, FILESIZE);

    /* Write directly to memory */
    const char *msg = "OSSP mmap() write demonstration: Direct memory write reflected to disk!";
    strncpy(mapped, msg, FILESIZE - 1);

    /* Synchronize mapped memory to disk synchronously */
    if (msync(mapped, FILESIZE, MS_SYNC) < 0) { perror("msync"); }
    printf("[mmap Write] msync() completed: Memory buffers flushed to persistent storage.\n");

    munmap(mapped, FILESIZE);
    close(fd);

    printf("[mmap Write] Verified disk file contents:\n");
    system("cat mmap_write_output.txt; echo");
    return 0;
}
```

Output screenshot:

```
teja27@TEJA:~$ nano exp10_task5.c
teja27@TEJA:~$ gcc exp10_task5.c -o exp10_task5
teja27@TEJA:~$ ./exp10_task5
OSSP mmap() write demonstration: Direct memory write reflected to disk!
[mmap Write] Memory mapped at address 0x7cb93209b000 with size 128
[mmap Write] msync() completed: Memory buffers flushed to persistent storage.
[mmap Write] Verified disk file contents:
teja27@TEJA:~$
```